$ spring run DomainLab --defense=0..100

# Spring Boot × DDD

// 從貧血模型到完整領域防線

| 0% | 25% | 50% | 75% | 100% |
| --- | --- | --- | --- | --- |
| 貧血模型 | Value Object | 充血實體 | Aggregate | Event / BC |

$ cat defense-plan.md

# 今日路線：一個訂單模型的五道防線

**0% / 病灶盤點**
魔法字串、規則散落、非法狀態

**25% / 值物件**
非法值建不出來

**50% / 充血實體**
非法狀態不可能存在

**75% → 100% / 邊界**
Aggregate、Event、Bounded Context

目標：每往右一格，就少一類「資料怎麼會變成這樣」的靈異事件。

PART 01 — 0%

# 貧血模型

Entity 只是資料袋，規則沒有家

# Transaction Script 的日常

```java
@Entity
public class Order {   // 只有資料，沒有行為
    private String status; // 魔法字串
    private double total;  // 精度看運氣
    // getter / setter 全開
}

@Service
public class OrderService { // 規則全塞在這
    public void pay(Long id, double amount) {
        Order order = repo.findById(id).get();
        if (!"PENDING".equals(order.getStatus()))
            throw new IllegalStateException();
        order.setStatus("PAID");
        order.setPaidAmount(amount); // 負數也照收
        emailService.send(order);
        repo.save(order);
    }
}
```

SMELL REPORT

- × 「何時能付款」的規則寫在 Service
- × 同一條規則在 12 個地方複製漂移
- × `setStatus("SHIPPED")` 誰都能呼叫
- × PM 說「付款」，程式寫 `setStatus`

規則沒有家 → 非法狀態隨時出現

PART 02 — 25%

# Value Object

非法值，從此建不出來

# 原始型別 → 值物件

```java
@Entity
public class Order {
    @Enumerated(EnumType.STRING)
    private OrderStatus status; // 打錯直接編譯不過

    @Embedded
    private Money total;        // 負數金額不存在
}
```

```java
  public record Money(BigDecimal amount, Currency currency) {
      public Money {
          if (amount.signum() < 0)
              throw new IllegalArgumentException("金額不可為負");
      }
  }
```

驗證只寫一次、寫在型別裡 — 不是「每次記得檢查」，是「非法值不存在」。

PART 03 — 50%

# 充血模型與領域語言

行為搬回實體，程式碼說業務的話

# 實體自我守護

```java
@Entity
public class Order {
    @Enumerated(EnumType.STRING)
    private OrderStatus status;
    @Embedded private Money total;

    protected Order() {} // 只給 JPA 用

    public void pay(Money amount) {
        if (status != OrderStatus.PENDING)
            throw new OrderNotPayableException(id);
        if (!amount.equals(total))
            throw new PaymentMismatchException();
        this.status = OrderStatus.PAID;
    }

    // 沒有 setStatus()
}
```

## Service 變薄

```java
@Service
public class OrderService {
    @Transactional
    public void pay(Long id, Money amount) {
        Order order = findOrder(id);
        order.pay(amount);
        // 找到人，請他做事
    }
}
```

**Ubiquitous Language**

PM 說「訂單付款」，程式碼寫 order.pay() —— 翻譯損耗歸零

狀態轉移必經守衛 → 非法狀態不可能存在

# 測試也跟著解放

```java
@Test
void 已出貨的訂單不能取消() {
    Order order = shippedOrder(); // 純物件，不用 @SpringBootTest

    assertThatThrownBy(order::cancel)
        .isInstanceOf(OrderAlreadyShippedException.class);
}
```

**規則只有一份**
住在它所屬的物件裡，改一處全站生效

**測規則不碰 DB**
領域邏輯 = 純物件行為，毫秒級驗證

# 動手拉拉看

$ spring run DomainLab

● 危險

## 0% 貧血模型

0% 25% 50% 75% 100%

- ✗ 金額用 double、狀態用魔法字串
- ✗ 商業規則散落 N 個 Service，改不齊
- ✗ setter 全開，狀態隨人改壞
- ✗ 想改哪張表就改哪張，一致性看緣分
- ✗ 跨模組直接互呼，改一處動全身

PART 04 — 75%

# Aggregate 與 Repository

一致性，終於有了邊界

# 聚合根是守門人

```java
@Entity
public class Order { // Aggregate Root
    @OneToMany(cascade = ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>(

    public void addItem(Product product, int qty) {
        if (status != OrderStatus.PENDING)
            throw new OrderLockedException(id);
        items.add(OrderItem.of(product, qty));
        this.total = calculateTotal(); // 不變條件
    }

    public List<OrderItem> items() {
        return List.copyOf(items); // 唯讀副本
    }
}
```

AGGREGATE RULES

1. 想動邊界內的東西 → 跟 root 說
2. 一個聚合 = 一個交易
（@Transactional 對齊）
3. 跨聚合只用 ID 參照，不抱整個物件圖

Repository 只服務聚合根：
沒有 OrderItemRepository

PART 05 — 100%

# Domain Event 與 Bounded Context

模組之間，只透過事件說話

# 宣告「發生了什麼」

```java
@Entity
public class Order
    extends AbstractAggregateRoot<Order> {

    public void pay(Money amount) {
        ensurePayable(amount);
        this.status = OrderStatus.PAID;
        registerEvent(new OrderPaid(id, total));
    }
}
```

**save() 時自動發布**
@TransactionalEventListener 保證 commit 成功後才處理

# 別的模組自己決定要不要關心

```java
@Component
class ShippingHandler {
    @TransactionalEventListener
    void on(OrderPaid event) {
        shippingService.prepare(event.orderId());
    }
}

@Component
class NotificationHandler {
    @TransactionalEventListener
    void on(OrderPaid event) {
        emailService.sendReceipt(event.orderId());
    } // 訂單模組完全不認識它
}
```

# Bounded Context：同名不同模型




同一個「Product」在不同上下文是不同的模型 — 這不是重複，是解耦。不再用 50 個欄位的萬能 Entity 撐全站。

# 0% vs 100%

| 指標 | 0% | 100% |
| --- | --- | --- |
| 非法值 | 負數金額、錯字狀態照收 | Value Object：建構即驗證 |
| 非法狀態 | setter 全開，隨人改 | 狀態轉移必經守衛 |
| 規則位置 | 散落 N 個 Service | 唯一的家，在所屬物件裡 |
| 一致性 | 看緣分 | Aggregate 交易邊界內恆成立 |
| 業務語意 | setStatus("PAID") 要人腦翻譯 | order.pay() 與 PM 同語言 |
| 模組耦合 | 改一行，全站重測 | Event + Bounded Context 分家 |

# 常見疑慮

**JPA Entity 當領域模型，會被綁死嗎？**

務實折衷可接受：protected 建構子給 JPA、@Embedded 放值物件，領域行為照寫。完全隔離是 100% 之後的選修題。

**跟 @Valid 是什麼關係？**

@Valid 守輸入格式（DTO），值物件與實體守商業規則。前者擋門口，後者守核心，兩層都要有。

**小專案也要 100%？**

每一格都是穩定停靠點：CRUD 後台停在 25% 就划算；有真實商業規則的核心模組至少到 50%。

# 今日重點

**25%**
值物件：非法值建不出來

**50%**
充血實體：非法狀態不可能存在

**75%**
Aggregate：一致性有邊界

**100%**
Event / BC：模組只透過事件說話

完美不是「畫滿架構圖」，是每條規則有唯一的家。

# Defense complete.

下一步：互動版文章 /springboot/ddd-in-spring-boot

```
$ spring run DomainLab --defense=100
value object → entity → aggregate → event
```